Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 11 - Coordinate Transforms | EX19

Goal: introduce coordinate transformations using rotation matrices. We start in 2D to clearly understand geometry, matrix multiplication, and inverse transformation.

Path

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/
    `-- class_11_coordinate_transforms/
        `-- exercises/
            `-- ex19_coordinate_rotation.py
```

Exercise convention

We use an **active rotation**: the coordinate frame remains fixed while the vector rotates. A positive angle rotates counter-clockwise.

Reference values

Case	Vector	Angle	Expected result
A	[100.0, 0.0]	90 deg	[0.0, 100.0] approx.
B	[3.0, 4.0]	30 deg	[0.5981, 4.9641] approx.
Norm B	||[3,4]||	-	5.0 before and after

BLOCK A - Degrees to radians

```
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
```

BLOCK B - Build the rotation matrix

```
def rotation_matrix_2d(angle_deg):
    theta = np.deg2rad(angle_deg)
    R = np.array([
        [ np.cos(theta), -np.sin(theta)],
        [ np.sin(theta),  np.cos(theta)]
    ])
    return R
```

BLOCK C - Rotate a vector

```
def rotate_vector_2d(vector, angle_deg):
    R = rotation_matrix_2d(angle_deg)
    return R @ vector
```

@ performs matrix multiplication. R has shape (2,2), vector has shape (2,), and the result has shape (2,).

BLOCK D - Reference case A

```
vector_A = np.array([100.0, 0.0])
rotated_A = rotate_vector_2d(vector_A, 90.0)
# expected approx [0.0, 100.0]
```

A theoretical zero may appear as a tiny floating-point value such as 6e-15.

BLOCK E - Vector norm

```
norm_before = np.linalg.norm(vector_A)
norm_after = np.linalg.norm(rotated_A)
same_norm = np.allclose(norm_before, norm_after)
```

BLOCK F - Inverse transformation

```
R = rotation_matrix_2d(30.0)
vector_B = np.array([3.0, 4.0])
rotated_B = R @ vector_B
recovered_B = R.T @ rotated_B
```

For an orthogonal rotation matrix, the inverse equals the transpose: R^-1 = R.T.

BLOCK G - Consecutive rotations

```
v1 = rotate_vector_2d(vector_B, 20.0)
v2 = rotate_vector_2d(v1, 40.0)
v_total = rotate_vector_2d(vector_B, 60.0)
np.allclose(v2, v_total)
```

BONUS - 3D rotation about Z

```
def rotation_matrix_z(angle_deg):
    theta = np.deg2rad(angle_deg)
    return np.array([
        [np.cos(theta), -np.sin(theta), 0.0],
        [np.sin(theta),  np.cos(theta), 0.0],
        [0.0,            0.0,           1.0]
    ])

vector_3d = np.array([10.0, 0.0, 5.0])
rotated_3d = rotation_matrix_z(90.0) @ vector_3d
# expected approx [0.0, 10.0, 5.0]
```

FINAL VALIDATION

```
Explain in comments:
- What does np.deg2rad() receive and return?
- What are the shape and meaning of R in 2D?
- Why do we use @ instead of * for R and vector?
- Why should vector norm remain unchanged after a rotation?
- What does R.T mean?
- Why can R.T recover the original vector?
- Why can a theoretical zero appear as a tiny non-zero float?
- What is the difference between rotating a vector and changing the coordinate frame?
```

Completion criterion

EX19 is complete when you can build a 2D rotation matrix, apply it with @, verify norm preservation, and recover the original vector using the inverse transformation.